AI Engineering and Claude Architect · Part IV, Security

AI Security, Lesson 4

PII, Tenancy, RBAC, Secrets and Audit

One customer asked about refunds, and got another company's refund policy, because the cache key was the question. Data escapes through the parts nobody thinks of as the AI.

Module	AI09 of 13
Lesson	4 of 5
Prerequisites	AI09 L1–L3, AI04 L4, AI05 L3, AI08 L8
Exam relevance	Mostly out of scope. Supports TS 2.3, 2.4, 5.6
Reading time	~35 minutes

Contents

1. PII, Tenancy, RBAC, Secrets and Audit

Lesson 4 of 5 · AI Security

PII, Tenancy, RBAC, Secrets and Audit

Key idea

The model is one stop on a long route for your data. Draw every place a prompt goes: logs, traces, caches, embeddings, memory, vendors and evaluation sets. Then apply tenant and role checks in code before retrieval, and treat what can't be detected reliably as a reason to send less.

> *Before: threat modelling, metadata filtering, the banking design. Teaches: data flow mapping, detection limits and tokenizing, tenant isolation, role checks for agents, secrets, tamper-evident audit, and deletion. Used by: AI10.*

None of this is legal advice. Privacy law varies by country and sector, and needs your compliance team.

The hook: the helpful cache

An invented example. A multi-tenant support product adds a semantic cache to cut costs, and it works beautifully, until Globex's employee asks, "What's our refund policy?" and receives Acme's. Both companies asked the same question, and the cache key was the question.

Real output from `DataGuards.java`:

```
Cache keys:
  question only: both tenants get [What's our refund policy?]
  scoped:        [acme|support|What's our refund policy?]
                 [globex|support|What's our refund policy?]
```

The model did nothing wrong, and neither did retrieval, which had correct filters. **A cache sits in front of your access checks, so its key has to contain everything those checks depend on.** Here that's the tenant and the set of roles, because an answer built from finance documents mustn't be served to

someone without the finance role.

Intuition: following a patient's file

Imagine following a paper medical record around a hospital. It's locked in the records room, and that part's fine. But there's a photocopy at the nurses' station, a note on a whiteboard, a copy faxed to a lab, and last year's copy in a box in the basement. The lock on the records room isn't the problem.

Your model call is the records room. Now find the photocopies.

Mechanism: where data goes

Map the route

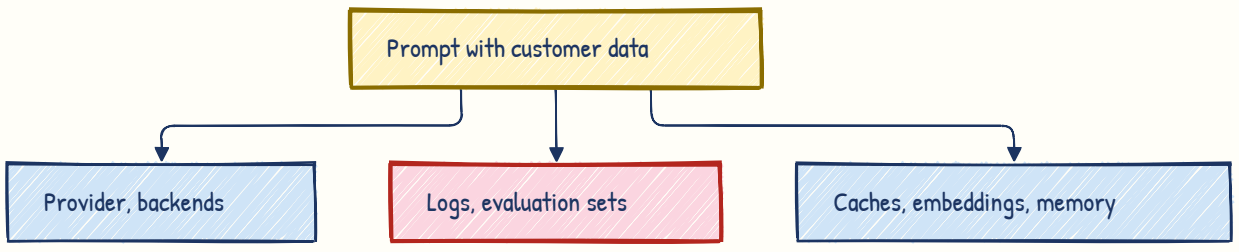


Figure 1. A single prompt can land in all of these. Each one needs its own access rule, and its own deletion rule.

I've highlighted logs and evaluation sets, which seem easiest to forget. That's my judgment, not a measurement. Debug logs often contain whole prompts. Evaluation sets get copied from production conversations and kept for years.

What happens at the provider

This changes, so read the current documents and your contract. Here's what Anthropic's API and data retention page says today:

Topic	What the page says
Training	Retained data is never used for model training without your express permission
Default	Conversation content isn't retained by default, with the exception of Covered Models
Covered Models	Fable 5.1, Mythos 5.1, Fable 5 and Mythos 5 require 30-day retention, and aren't available under ZDR unless expressly authorized
Zero data retention	By arrangement, per organization. Prompts and responses aren't stored at rest after the response returns
Features outside ZDR	Batch has 29-day retention. Files persist until deleted or expired. MCP connector and Skills follow standard policy. Managed Agents transcripts persist until you delete them
No block on those	Under ZDR, the API **doesn't block** ineligible features, so using one steps outside the arrangement for that data. Under HIPAA readiness, it returns a 400
Regardless	Flagged content may be retained for up to 2 years, and data may be retained where the law requires
Other platforms	On Bedrock and Google-managed Google Cloud, the cloud provider is the data processor

The "no block" row is an architecture trap. **An engineer adding Batch or the MCP connector may quietly take data outside your ZDR arrangement, and no error will tell them.** So check feature use in code review.

Prompt caching: the docs say caches are isolated between organizations, and also by workspace on the Claude API, Claude Platform on AWS and Microsoft Foundry. It's eligible for ZDR, with a qualification: cache representations and hashes are held in memory for the cache lifetime. Bedrock and Google-managed Google Cloud isolate by organization only. **This is the provider's cache.** It doesn't help with your own semantic cache from the hook, and a workspace isn't a customer boundary, so your tenants still rely on your code.

PII: detecting it, and its limits

Detection helps, and you shouldn't trust it. Real output, on invented data:

```
Detection, on invented data:
  card, valid check digit        [card]
  order number, same length      []
  email                          [email]
  api key                        [secret]
  password in config             [secret]
  name and address               []
  card number spelled out        []
  national ID                    []
```

Good results first: the Luhn check tells a card from a same-length order number, although one random number in ten still passes. Then read the misses, which I left in deliberately. A name and address have no pattern at all. Regex can't see them. A named-entity model catches more, and also misses some, and it needs measuring on your own data.

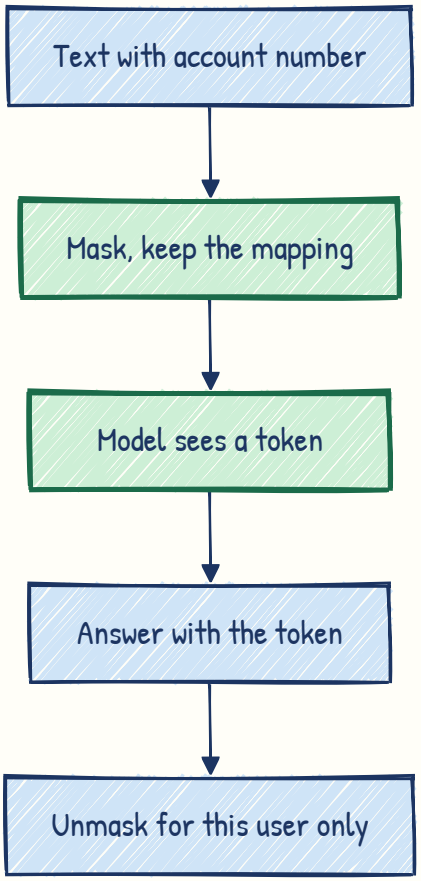


Figure 2. Tokenizing lets the model reason about a value it never sees. The real value returns only at the edge.

AI08 L8 built this as `PiiTokenizer`. It works for values the model needn't read. It's useless for tasks that depend on the content, like "is this name on the sanctions list?", and it needs a secure mapping store.

The stronger controls don't depend on detection at all:

Control	Why it's stronger
Don't fetch it	A tool returning only the fields needed can't leak the others
Don't send the whole record	Same reasoning, one layer down
Don't log prompts by default	Log IDs and sizes, and sample with care
Short retention everywhere	What isn't kept can't be breached

The best PII filter is a query that never selected the column.

Tenant isolation

The ladder, weakest to strongest. AI04 L4 taught filtering, and AI05 L3 applied it to retrieval, so here's only the security view:

Level	How	Risk
Shared index, filter after search	App removes other tenants' results	Bugs leak, and result counts collapse
Shared index, filter in the query	`WHERE tenant_id = ?`	A missing filter leaks everything
Row-level security	Database enforces it from a session variable	Stronger, as forgetting isn't possible, if set up properly
A namespace or schema per tenant	Separate logical stores	More operational cost
A database per tenant	Physical separation	Highest cost. For regulated or very large tenants

The rule that matters most: **the tenant ID comes from the verified session, and never from the model or the request body.** Anything the model supplies can be steered (L3).

Here's every shared thing to check, because tenants meet in more places than the index:

Shared thing	The leak	The fix
Vector index	Another tenant's chunk retrieved	Filter in the query, from the session
Semantic cache	Their answer served to you	Tenant and roles in the key
Agent memory	Their facts recalled in your session	Scope by tenant and user
Fine-tuning or shared few-shot examples	Their data in everyone's prompt	Never mix, or anonymize with review
Logs and dashboards	Support staff see all tenants	Access rules, and redaction
Evaluation sets	Their conversations in shared tests	Keep them per tenant, or use synthetic data
Shared uploads	One tenant poisons what others retrieve (L2)	Isolate, and label sources

RBAC: the agent's rights are an intersection

A common confusion is whether the agent has its own rights. It has both.

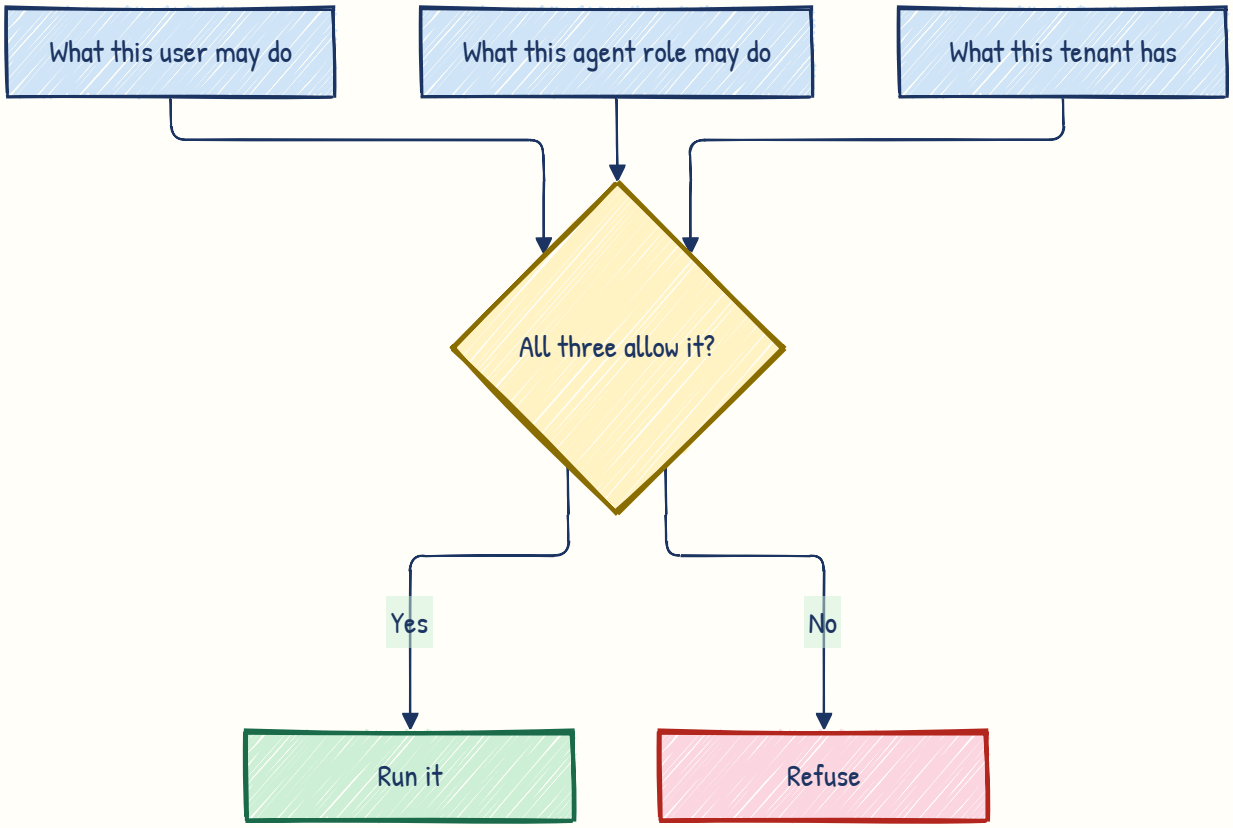


Figure 3. An action needs all three. Neither the agent's grant nor the user's rights alone are enough.

→ A finance manager using the wiki agent **can't** approve payments through it, because that agent has no payment tool.

→ A junior employee using the finance agent **can't** read payroll, because they personally can't.

Real output, with tenant checked first and denial as the default:

```
Access:
  asha reads acme-close     true
  asha reads acme-hr        false
  asha reads globex-close   false
```

Retrieval is a read, and needs the same check. If a user can't open a document, then the model mustn't quote it to them. A summary of a secret document is a leak of that document.

Permissions change. An index built last month may hold documents someone's lost access to. Check rights at query time from the live source, or accept a defined delay, and write that delay down.

Multi-agent systems. A coordinator's subagents should receive the user's identity, and no broader. It's easy to lose this when an agent calls a service with its own credentials (L3's confused deputy).

Secrets

Rule	Why
Never put a secret in a prompt	It reaches logs, caches and possibly output. OWASP lists prompt leakage
Keep it out of the sandbox	L3's proxy pattern
Keep it out of repositories	`${VAR}` in `.mcp.json` (TS 2.4). CLAUDE.md is committed text
Scrub tool output	A tool that returns a config file can return a password
Keep tool errors generic	Frameworks leak exception text by default (AI07). `AgentService` doesn't, and its test uses a database error containing a host
Rotate, and scope	Assume one leaks. One key per service
Scan	Catches known formats, in commits and logs

Scanning shows the same limits as PII detection. It found `sk-ant-` and `password =` above. A secret in an unknown format passes through.

Audit: what you'll need on the worst day

When something goes wrong, you'll want to reconstruct what happened. For an agent, that's more than a request log:

Record	Why
User, tenant, session	Who
Model ID, prompt version, tool versions	What produced this behaviour
Every tool call, with arguments, decision and result status	What it did. `ToolLedger` does this
Which untrusted content was read, by source	Tracing an injection back. Often missing
Policy decisions and approvals, with who approved	Who decided
Tokens and cost	Abuse and runaway loops

Make it tamper-evident. Real output from a small hash chain:

```
Audit log:
  intact, first broken entry: -1
  edited, first broken entry: 1
  chain rewritten, first broken entry: -1
  head still matches the copy kept elsewhere: false
```

Line two: an edit breaks the chain. **Line three is the honest part. Someone with full write access can recompute every hash, and the chain looks fine.** Line four: that's only caught if you've kept the latest hash somewhere the attacker can't reach. **A hash chain without an external anchor proves little against an insider.**

The chain has a cost as well: it detects tampering, and doesn't prevent it. Append-only storage or write-once object storage addresses that, and it's your platform's job to provide them.

Audit logs conflict with privacy. They're valuable because they keep everything, and privacy says not to. An engineering recommendation: store references and hashes of sensitive content, keep the content itself in a separately protected store with short retention, and write down the retention decision.

Deletion: where's the data really?

A customer asks you to delete their data. Here's the list:

Store	How deletion works	Catch
Source database	Delete	The easy part
Vector index	Delete chunks by document ID (AI05 L7)	Needs provenance to find them
Caches	Expire, or key by user	Your semantic cache may hold derived answers
Agent memory and summaries	Delete by user	Summaries mix several sources
Logs and traces	Retention expiry	Can you search for them by user?
Evaluation sets	Remove	Often copied by hand
Provider	Per the retention terms	Files and Managed Agents need explicit deletion
Backups	Expiry	Know your window
A fine-tuned model	You can't practically remove it	Don't train on data you may have to delete

Provenance is what makes this possible. Without a link from a chunk to its source, you can't find it to delete it.

Can embeddings reveal their source text? Treat them as personal data if the source was. I haven't read or tested a specific reconstruction result for this course, so that's a cautious recommendation and not a measured claim.

Why the obvious fixes fall short

Obvious idea	Why it falls short
A regex redactor is enough	Missed names, addresses and an ID above
ZDR, so we're safe	It covers that provider's storage, for eligible features only. Your logs are yours
Filter results after retrieval	One bug leaks, and it's after the data has been fetched
The model is told not to reveal it	Advice, not access control
An audit log in the same database	Whoever breaks in can edit it
A shared service account, with an app check	One missed check is a total leak
Delete it from the source	Copies live on in six other stores

How Anthropic approaches it

Topic	What's published
Retention	The API and data retention page, with per-feature eligibility
Credentials	Vaults for Managed Agents, proxy for Claude Code on the web (L3)
Claude Code	Credential storage, managed settings, OpenTelemetry, audit logging in cloud sessions
Multi-tenant RAG	**Nothing I've read describes how Anthropic isolates tenants internally**, so the isolation ladder above is general practice

How the field approaches it. OWASP lists Sensitive Information Disclosure as LLM02, and Vector and Embedding Weaknesses as LLM08. Row-level security and namespaces are long-standing database practice.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: the app passes a tenant ID to each query, and logs everything.

Improved: identity from the session, filtering in the query, and redaction in logs. New problem: each new store is a new place to forget the rule.

Production: a data plane where identity travels in a context object, and stores enforce it.

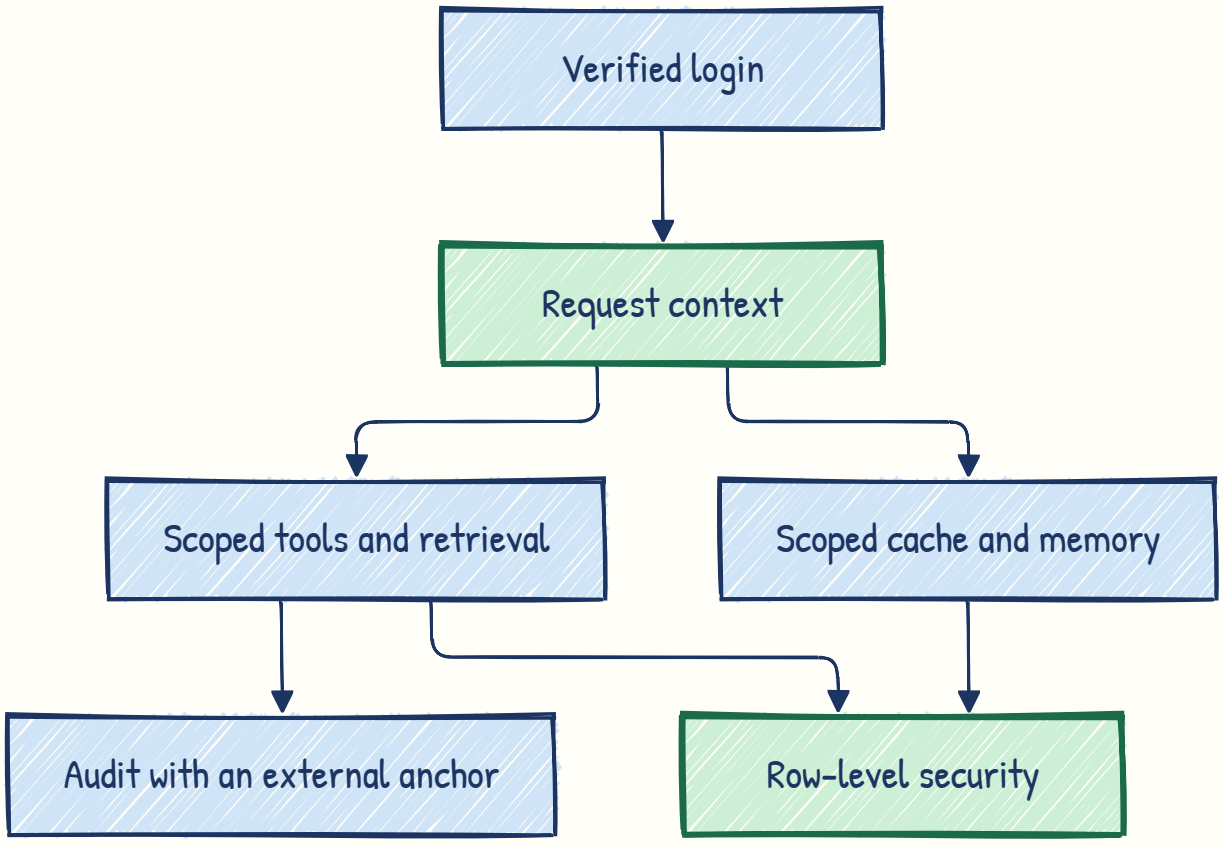


Figure 4. Identity is set once, at the edge. Every store takes it from the context, and no store takes it from the model.

Component	Why it exists	How it communicates
Request context	One source of identity	Passed in code. Never via the prompt (`AgentTool.Context`)
Field-level tools	The query doesn't select what's not needed	Service API
Row-level security	The store refuses, even when the app forgets	Database session variable
Scoped cache	Prevents the hook's leak	Key includes tenant and roles
PII gateway	Masks what it can recognize, before the model	In-process or a service
Log redaction	Keeps prompts out of general logs	Logging pipeline
Deletion service	One workflow covers all stores	Job per store, with a report
Audit store	Tamper-evident, separately controlled	Append-only, with an external anchor

Failures:

Failure	How you'd notice	What to do
Cross-tenant answer	A test with two tenants, or a customer	Abuse-case test on every shared store. Treat as an incident
PII in logs	A scanner on the log stream	Purge, fix, and rotate if secrets
Stale permissions in the index	Audit of a removed user	Check live at query time
Redactor breaks a task	Evaluation drops	Measure, and exclude that field from redaction, deliberately
Deletion misses a store	Quarterly test deletion	Maintain the store list
A feature takes data outside ZDR	Code review, config scan	Allowlist features

Java

Two checks, each a single decision.

```
return user.tenant().equals(doc.tenant())
        && user.roles().contains(doc.requiredRole());
```

Deny by default: both must hold. There's no path where a missing role means yes.

```
return tenant + "|" + roles + "|" + question;
```

Everything the access decision depends on goes in the key. Roles should be sorted and normalized in a real system, and a real semantic cache has to apply the same scope as a filter on its vector lookup, not only in a string.

```
String previous = entries.isEmpty() ? "start" : entries.getLast()[1];
entries.add(new String[] {event, hash(previous + event)});
```

Each hash covers the previous one, so changing an old entry breaks all later ones. A real log uses a keyed hash or a signature, along with an external anchor.

Trade-offs

Decision	Benefit	Cost
Tokenize PII	Provider doesn't see values	Mapping store, and some tasks fail
Don't log prompts	Far less exposure	Harder debugging. Sample with care
Tenant in the cache key	No leak	Lower hit rate
A database per tenant	Strong isolation	Cost and operations
Live permission checks	Accurate	Latency, and dependence on another system
ZDR	Less provider retention	Excludes some features and some models
Full audit	Reconstruction	Storage, and it's a sensitive store in itself

When not to go this far: a single-tenant internal tool on non-sensitive data needs identity and basic logging, not a tokenizer or per-tenant databases.

Common mistakes

Forgetting caches and memory in a tenant review. Reviews tend to look at retrieval. Test each store with two tenants and the same question.

Common mistakes

Copying production conversations into an evaluation set. They're personal data in a new place, with no retention rule, and often in a repository.

What to say in an interview

"I'd map every store a prompt reaches: logs, caches, embeddings, memory, evaluation sets, and the provider. Identity comes from the session and goes in every key and every filter, and row-level security backs it up. An action needs the user's rights, the agent's role, and the tenant to all agree. Detection misses things, so the main control is not fetching the data. And audit records which untrusted content was read."

Certification lens

How this lesson maps to CCAR-F

Mostly out of scope. Tenancy, PII and retention aren't exam topics. Nearby: scoped tools (TS 2.3), secrets in `.mcp.json` (TS 2.4), and provenance (TS 5.6). Don't expect retention figures on the exam, and don't memorize them for real work: they change, so read the page.

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Multi-Agent Research · Medium. A synthesis agent must let readers check each claim, and later the team must be able to find every report built on a withdrawn document. What should subagents return?

A. A prose summary, with the source named in the final paragraph

B. Structured claims, each mapped to its source document, location and date

C. Only the final conclusions, to save tokens

D. Links to the search engine queries they ran

A **B** (TS 5.6). Provenance has to stay attached through the pipeline. A loses the claim-level link, C removes it, and D doesn't identify which document supports which claim.

Q Q2 · Your organization has zero data retention. A developer adds the Batch API to reduce cost. What's the concern, and how would you catch it?

A Per the retention page, Batch isn't ZDR-eligible, with 29-day retention, and under ZDR the API doesn't block ineligible features. So data leaves the arrangement with no error. Catch it with code review or a config scan of an allowlist of features, and decide deliberately.

Q Q3 · Name five places other than retrieval where two tenants' data can meet.

A Semantic cache, agent memory, logs and dashboards, evaluation sets, shared few-shot examples or fine-tuning data, and shared uploads. Any two or more of these show the point: each is a store that needs the tenant in its key or filter.

Q Q4 · A manager with payment rights uses the wiki agent, and asks it to approve a payment. What should happen, and why?

A It should refuse, since the wiki agent has no payment tool. An action requires the user's rights and the agent's role and the tenant to agree. The user's rights alone don't extend the agent.

Revision - Lesson 4

→ Map every store a prompt reaches. Logs and evaluation sets are easily forgotten.

→ A cache sits in front of access checks, so its key needs tenant and roles.

→ Provider retention varies by feature. Under ZDR, ineligible features aren't blocked.

→ Provider cache isolation doesn't protect your own tenants from each other.

→ Regex misses names, addresses, and unfamiliar IDs. The best filter is not selecting the column.

→ Tenant ID comes from the session, never from the model.

→ Rights are an intersection of user, agent role and tenant. Retrieval is a read, and needs the check.

→ Secrets stay out of prompts, sandboxes and repositories. Keep tool errors generic.

→ Audit records which untrusted content was read. A hash chain needs an external anchor.

→ Deletion spans many stores, and provenance makes it possible.

Your notes